

FULL STACK WEB DEVELOPMENT

ASSIGNMENT – 1

IMPLEMENT INHERITANCE TYPES USING JAVASCRIPT

Submitted By

Ch Venu Gopal Reddy

Roll Number: A24126552078

Subject

Full Stack Web Development

Academic Assignment

JavaScript – Types of Inheritance

Introduction

Inheritance is an important concept of Object-Oriented Programming (OOP). It allows one class to acquire the properties and methods of another class. In JavaScript, class inheritance is commonly implemented using the extends keyword and the super() method.

This assignment demonstrates five commonly discussed inheritance types using the Shape concept: Single, Multilevel, Hierarchical, Multiple, and Hybrid inheritance.

Important JavaScript Concepts Used

class – used to define a class.

constructor() – used to initialize object properties.

extends – used to create inheritance between classes.

super() – used to call the parent class constructor.

Object.assign() – used to add mixin functionality.

console.log() – used to display output in the terminal.

1. Single Inheritance

Single inheritance means one child class inherits properties and methods from one parent class. In this example, Circle inherits from Shape.

Inheritance Structure:

Shape → Circle

File Name: singleInheritance.js

Program:

```
// Single Inheritance
// Shape → Circle

class Shape {
  constructor(color) {
    this.color = color;
  }

  displayColor() {
    console.log("Shape Color:", this.color);
  }
}

class Circle extends Shape {
  constructor(color, radius) {
    super(color);
    this.radius = radius;
  }

  calculateArea() {
    return Math.PI * this.radius * this.radius;
  }
}
```

```

    }
}

const circle = new Circle("Red", 5);

console.log("----- SINGLE INHERITANCE -----");
circle.displayColor();
console.log("Circle Radius:", circle.radius);
console.log("Circle Area:", circle.calculateArea().toFixed(2));

```

Expected Output:

```

----- SINGLE INHERITANCE -----
Shape Color: Red
Circle Radius: 5
Circle Area: 78.54

```

2. Multilevel Inheritance

Multilevel inheritance occurs when a class inherits from another derived class. Polygon inherits from Shape and Rectangle inherits from Polygon.

Inheritance Structure:

```
Shape → Polygon → Rectangle
```

File Name: multilevelInheritance.js

Program:

```

// Multilevel Inheritance
// Shape → Polygon → Rectangle

class Shape {
    constructor(color) {
        this.color = color;
    }

    displayColor() {
        console.log("Shape Color:", this.color);
    }
}

class Polygon extends Shape {
    constructor(color, sides) {
        super(color);
        this.sides = sides;
    }

    displaySides() {
        console.log("Number of Sides:", this.sides);
    }
}

class Rectangle extends Polygon {
    constructor(color, sides, length, width) {

```

```

        super(color, sides);
        this.length = length;
        this.width = width;
    }

    calculateArea() {
        return this.length * this.width;
    }
}

const rectangle = new Rectangle("Blue", 4, 10, 5);

console.log("----- MULTILEVEL INHERITANCE -----");
rectangle.displayColor();
rectangle.displaySides();
console.log("Rectangle Area:", rectangle.calculateArea());

```

Expected Output:

```

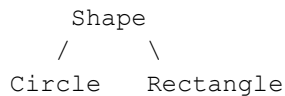
----- MULTILEVEL INHERITANCE -----
Shape Color: Blue
Number of Sides: 4
Rectangle Area: 50

```

3. Hierarchical Inheritance

Hierarchical inheritance occurs when multiple child classes inherit from the same parent class. Both Circle and Rectangle inherit from Shape.

Inheritance Structure:



File Name: hierarchicalInheritance.js

Program:

```

// Hierarchical Inheritance

class Shape {
    constructor(color) {
        this.color = color;
    }

    displayColor() {
        console.log("Shape Color:", this.color);
    }
}

class Circle extends Shape {
    constructor(color, radius) {
        super(color);
        this.radius = radius;
    }
}

```

```

        calculateArea() {
            return Math.PI * this.radius * this.radius;
        }
    }

    class Rectangle extends Shape {
        constructor(color, length, width) {
            super(color);
            this.length = length;
            this.width = width;
        }

        calculateArea() {
            return this.length * this.width;
        }
    }

    const circle = new Circle("Green", 3);
    const rectangle = new Rectangle("Yellow", 6, 4);

    console.log("----- HIERARCHICAL INHERITANCE -----");

    console.log("Circle:");
    circle.displayColor();
    console.log("Circle Area:", circle.calculateArea().toFixed(2));

    console.log("\nRectangle:");
    rectangle.displayColor();
    console.log("Rectangle Area:", rectangle.calculateArea());

```

Expected Output:

```

----- HIERARCHICAL INHERITANCE -----
Circle:
Shape Color: Green
Circle Area: 28.27

Rectangle:
Shape Color: Yellow
Rectangle Area: 24

```

4. Multiple Inheritance

JavaScript classes do not directly support multiple inheritance. Similar behavior can be implemented using mixins. In this example, GraphicShape inherits from Shape and receives the draw() method from the Drawable mixin.

Inheritance Structure:

```
Shape + Drawable → GraphicShape
```

File Name: multipleInheritance.js

Program:

```
// Multiple Inheritance using Mixins
// Shape + Drawable → GraphicShape

class Shape {
  constructor(color) {
    this.color = color;
  }

  displayColor() {
    console.log("Shape Color:", this.color);
  }
}

const Drawable = {
  draw() {
    console.log("Shape is drawn successfully.");
  }
};

class GraphicShape extends Shape {
  constructor(color) {
    super(color);
  }
}

Object.assign(GraphicShape.prototype, Drawable);

const graphicShape = new GraphicShape("Purple");

console.log("----- MULTIPLE INHERITANCE -----");
graphicShape.displayColor();
graphicShape.draw();
```

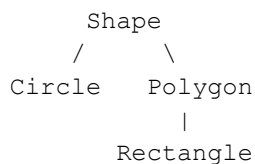
Expected Output:

```
----- MULTIPLE INHERITANCE -----
Shape Color: Purple
Shape is drawn successfully.
```

5. Hybrid Inheritance

Hybrid inheritance is a combination of two or more inheritance types. This example combines hierarchical and multilevel inheritance.

Inheritance Structure:



File Name: hybridInheritance.js

Program:

```
// Hybrid Inheritance

class Shape {
    constructor(color) {
        this.color = color;
    }

    displayColor() {
        console.log("Shape Color:", this.color);
    }
}

class Circle extends Shape {
    constructor(color, radius) {
        super(color);
        this.radius = radius;
    }

    calculateArea() {
        return Math.PI * this.radius * this.radius;
    }
}

class Polygon extends Shape {
    constructor(color, sides) {
        super(color);
        this.sides = sides;
    }

    displaySides() {
        console.log("Number of Sides:", this.sides);
    }
}

class Rectangle extends Polygon {
    constructor(color, sides, length, width) {
        super(color, sides);
        this.length = length;
        this.width = width;
    }

    calculateArea() {
        return this.length * this.width;
    }
}

const circle = new Circle("Orange", 4);
const rectangle = new Rectangle("Pink", 4, 8, 6);

console.log("----- HYBRID INHERITANCE -----");

console.log("Circle:");
circle.displayColor();
console.log("Circle Area:", circle.calculateArea().toFixed(2));
```

```
console.log("\nRectangle:");
rectangle.displayColor();
rectangle.displaySides();
console.log("Rectangle Area:", rectangle.calculateArea());
```

Expected Output:

```
----- HYBRID INHERITANCE -----
Circle:
Shape Color: Orange
Circle Area: 50.27

Rectangle:
Shape Color: Pink
Number of Sides: 4
Rectangle Area: 48
```

How to Run the Programs in VS Code

1. Create a folder named assignment-01-Inheritance and a sub folder src to store source files.
2. Create the following five JavaScript files inside the sub folder:
 - singleInheritance.js
 - multilevelInheritance.js
 - hierarchicalInheritance.js
 - multipleInheritance.js
 - hybridInheritance.js
3. Open the folder in Visual Studio Code.
4. Open the integrated terminal using Terminal → New Terminal.
5. Run the required JavaScript file using Node.js.

```
node singleInheritance.js
```

```
node multilevelInheritance.js
```

```
node hierarchicalInheritance.js
```

```
node multipleInheritance.js
```

```
node hybridInheritance.js
```

Note

Make sure Node.js is installed on the computer before running the programs. The output of each program will be displayed in the VS Code terminal.

Conclusion

The different types of inheritance were successfully implemented using JavaScript. Single, multilevel, hierarchical, multiple, and hybrid inheritance were demonstrated using independent programs based on the Shape concept.